

Constant-Memory Retrieval via Koopman Operator Estimation for Mamba-3

Anonymous Author(s)

Abstract

Long chain-of-thought reasoning and agentic tool-calling produce traces spanning tens of thousands of tokens, yet Transformer KV caches grow linearly with sequence length, creating a memory bottleneck on commodity hardware. State-space models offer constant-memory recurrence but suffer a *memory cliff*: retrieval accuracy collapses once the gap between a stored fact and its query exceeds the effective horizon of the recurrent state. We introduce **Spectral Koopman Attention (SKA)**, a module that augments SSM layers with a closed-form dynamical operator whose sufficient statistics are accumulated in constant memory with no KV cache, nor attention layers. SKA fits a spectral linear system to the key, value history via kernel ridge regression and retrieves through a learned power-iterated filter, all from $O(r^2)$ streaming state where r is a small projection rank. On a Tool Trace benchmark requiring retrieval of high-entropy identifiers after distractor gaps of up to 8,192 tokens, a bare Mamba-3 SSM fail to exceed chance accuracy ($\sim 2\text{--}8\%$), while SKA-augmented models reach near perfect retrieval accuracy. Ablations confirm that the spectral operator, not the masking strategy, drives the retrieval gain.

CCS Concepts

• Computing methodologies;

Keywords

State Space Models (SSM), Spectral Koopman Attention (SKA), Memory Cliff, Koopman Operator Theory, Clebsch-Gordan Feature Lift, Constant-Memory Retrieval, Associative Recall, Mamba

ACM Reference Format:

Anonymous Author(s). 2026. Constant-Memory Retrieval via Koopman Operator Estimation for Mamba-3. In *Proceedings of Conference on AI and Agentic Systems (CAIS '26)*. ACM, New York, NY, USA, 16 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

State space models (SSMs) have emerged as a compelling alternative to Transformers for sequence modeling, offering linear-time inference and constant memory per decoding step. Recent architectures such as Mamba [12], Mamba-2 [10], and Mamba-3 [11] have closed the quality gap with attention-based models on language modeling benchmarks while retaining the efficiency advantages of recurrent

computation. However, a fundamental tension remains: SSMs compress the entire history of a sequence into a fixed-size hidden state, and this compression is lossy. When a downstream task requires retrieving a specific fact deposited hundreds or thousands of tokens earlier, the recurrent state may no longer contain it. We call this failure mode the *memory cliff*: beyond a task-dependent horizon, retrieval accuracy collapses to chance regardless of model capacity.

The memory cliff is especially consequential in agentic settings, where a model must store structured tool outputs across long stretches of intervening reasoning and retrieve them accurately when composing a final answer. Transformers handle this through attention, which provides content-addressed lookup over all prior tokens at the cost of a KV cache that grows linearly with sequence length, often dominating inference memory in production deployments. SSMs pay no such memory cost, but they have no mechanism for *selective* retrieval from the past. The recurrent state is a summary, not an index.

We introduce **Spectral Koopman Attention (SKA)**, a module that gives SSM layers explicit long-range retrieval while preserving their constant-memory property. The key observation is that content-addressed retrieval, the operation that motivates KV caches, can be expressed as a kernel ridge regression whose sufficient statistics are a covariance matrix and a cross-covariance vector, both accumulated as running sums in $O(r^2 + rP)$ memory where r is a small projection rank and P is the value dimension. At query time, a single closed-form solve replaces the $O(T)$ inner product that attention computes against the full cache. The result is exact associative retrieval from a fixed-size streaming state, with no attention layers and no KV cache.

Concretely, SKA projects the SSM’s internal representations into a low-rank feature space, accumulates key-value covariance statistics as tokens stream past, and fits a whitened Koopman operator whose persistent spectral modes identify which bindings to retain. At query time, the operator retrieves stored values through a power-iterated spectral filter. The entire computation, feature projection, statistic accumulation, and retrieval, operates in constant memory per layer.

Our contributions are as follows:

- (1) We identify the *memory cliff* in SSMs and frame exact retrieval as kernel ridge regression over streaming sufficient statistics, yielding $O(r^2 + rP)$ memory independent of sequence length.
- (2) We propose SKA, a module that fits a spectral Koopman operator to accumulated covariance statistics and retrieves stored values through power-iterated filtering, requiring no attention layers and no KV cache.
- (3) We show that SKA-augmented SSMs achieve near-perfect retrieval on associative recall benchmarks where a bare SSMs fail to exceed chance, with no degradation as context length grows well beyond the training distribution.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CAIS '26, San Jose

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-XXXX-X/2018/06

<https://doi.org/XXXXXXX.XXXXXXX>

2 Related Works

2.1 State Space Models and the Mamba Family

Structured State Space Models. The foundations of modern SSMs trace back to S4 [13], which demonstrated that continuous-time state space formulations could effectively model long-range dependencies while maintaining computational efficiency. S4 introduced the HiPPO initialization for the state matrix, enabling unbounded context and efficient training via global convolutions. Subsequent work refined these models through diagonal parameterizations and improved initialization [14, 26].

Mamba. Gu and Dao [12] introduced selective state space models, making SSM parameters input-dependent to enable content-based filtering of information along the sequence. This selectivity, combined with a hardware-aware parallel scan, yielded linear-time inference with 5× higher throughput than comparably sized Transformers.

Mamba-2. Dao and Gu [10] established a duality between SSMs and structured variants of attention through the SSD framework, recasting the selective SSM computation as matrix multiplications amenable to tensor core acceleration and achieving 2–8× faster training than Mamba.

Mamba-3. Gu et al. [11] introduced a complex-valued state update rule, a more expressive recurrence, and a multi-input multi-output formulation that better exploits hardware parallelism during decoding, setting a new Pareto frontier for quality under fixed inference budgets.

2.2 Retrieval Limitations of SSMs

Despite their efficiency, SSMs exhibit well-documented retrieval failures that motivate our work.

Associative Recall. Arora et al. [6, 7] introduced the multi-query associative recall (MQAR) benchmark and showed that linear-time models face an inherent recall-throughput tradeoff. Jelassi et al. [16] formally proved that SSMs have strictly lower capacity than Transformers for copying long sequences owing to their fixed-size state. Recent work on joint recall [15] further demonstrates that context-dependent key-value retrieval requires state dimension to grow linearly with the number of stored entries.

Exponential Memory Decay. Wang et al. [29] showed that SSM layers are inherently biased toward local information, with relevance of distant tokens decaying exponentially. Needle-in-a-haystack evaluations confirm position-dependent retrieval degradation, particularly for tokens early in long contexts. This exponential decay is the direct failure mode that SKA is designed to circumvent.

In-Context Learning. Lieber et al. [19] demonstrated that pure Mamba models struggle with in-context learning, often failing to follow few-shot formats. Adding even a single attention layer out of eight restored ICL capabilities, suggesting that attention is critical for direct token-to-token interactions.

Mechanistic Differences. Interpretability studies [5] revealed that Mamba solves associative recall through its short convolution component rather than the SSM itself, highlighting the fragility of its retrieval capabilities.

2.3 Hybrid SSM-Attention Architectures

The retrieval limitations of pure SSMs have driven research into hybrid architectures. Jamba [19] interleaves Mamba and attention layers with a 7:1 ratio alongside mixture-of-experts modules. Hymba [23] runs attention and Mamba in parallel within each block. Gated Delta Networks [32] combine gating for adaptive memory erasure with the delta update rule for precise memory modifications. Waleffe et al. [28] validated the hybrid approach at 8B scale, showing that a large proportion of SSM layers with a small number of attention layers can surpass standard Transformers. The Nemotron family [24, 25] demonstrated this at production scale for agentic workloads, replacing 92% of attention layers with Mamba-2 blocks while matching or exceeding Transformer accuracy.

These architectures address SSM retrieval limitations by retaining some attention layers. SKA takes a different approach: rather than falling back on attention for retrieval, it augments the SSM layers themselves with an explicit retrieval mechanism that operates in constant memory.

2.4 KV Cache Compaction

A parallel line of work mitigates the Transformer memory bottleneck through KV cache compression. Token eviction strategies retain high-impact tokens based on attention scores [1, 18, 33]. Compression methods exploit cross-layer similarity or quantization [2, 20, 27]. Semantic-aware approaches operate on linguistic chunks rather than isolated tokens [3]. Hierarchical methods exploit the observation that attention concentrates in deeper layers [9, 31]. Dynamic Memory Sparsification [4] achieves 8× compression with minimal fine-tuning.

These are effective post-hoc mitigations, but they do not eliminate the fundamental linear growth of the cache. SKA sidesteps the problem entirely: its stored statistics are fixed-size regardless of sequence length, performing content-addressed retrieval from a constant-memory operator rather than a growing key-value store.

2.5 Koopman Operator Theory in Machine Learning

Koopman operator theory provides a framework for analyzing non-linear dynamical systems through linear operators acting on observable functions [17, 22]. In machine learning, Koopman methods have been applied to dynamical system identification and prediction [8, 21], where a learned observable basis and its associated linear operator capture the evolution of nonlinear state trajectories. Extended Dynamic Mode Decomposition (EDMD) [30] estimates the Koopman operator from data via least-squares regression on lifted features, the same mathematical structure that SKA adapts for sequence modeling. To our knowledge, SKA is the first application of Koopman operator estimation as a retrieval mechanism within a neural sequence model, using the operator’s persistent eigenmodes to perform content-addressed value lookup rather than dynamical forecasting.

2.6 Positioning of SKA

SKA occupies a distinct point in the design space. Unlike hybrid architectures that interleave SSM and attention layers, SKA augments SSM layers directly with a retrieval module that shares attention’s

content-addressed lookup capability but not its linearly growing memory cost. Unlike KV compaction methods that reduce an existing cache, SKA requires no cache to begin with. The closest methodological connection is to Koopman-based system identification, but SKA repurposes the operator for associative retrieval rather than forecasting, stabilized by a feature lift that accounts for the nonstationarity introduced by rotary embeddings in modern SSMs.

3 Method

We present a hybrid architecture that replaces both softmax attention and SwiGLU feedforward layers with Koopman operator-based alternatives, while retaining state-space models for local sequence processing. The result is a model with $O(1)$ inference state per layer—constant regardless of context length—making it suitable as a backbone for agentic systems that must process long tool outputs, multi-turn conversations, and retrieved documents under bounded memory.

We describe the three core components: the SSM backbone (§3.1), Spectral Koopman Attention for global retrieval (§3.2), and the Spectral Koopman MLP (§3.3). We then describe how they are composed into a complete language model (§3.4).

3.1 SSM Backbone

The majority of sequence layers use a selective state-space model for local, sequential processing. We support two SSM variants depending on the experimental setting.

Mamba-2 (language modeling). For from-scratch language model training, we use Mamba-2 [10] as the SSM backbone. Each Mamba-2 layer maintains a recurrent state $h_t \in \mathbb{R}^{N \times P}$ with state dimension N and head dimension P , updated via a hardware-efficient selective scan. The state evolves as:

$$h_t = \bar{A}_t h_t + \bar{B}_t x_t, \quad y_t = C_t h_t, \quad (1)$$

where $\bar{A}_t$, $\bar{B}_t$, and C_t are input-dependent parameters computed from learned projections. This provides $O(1)$ state per layer during autoregressive inference.

Mamba-3 (synthetic tasks). For controlled synthetic experiments (§4.1), we use Mamba-3 [?], which extends Mamba-2 with data-dependent rotary position embeddings and a trapezoidal integration rule that reduces discretization error. This variant enables us to isolate the contribution of SKA on tasks that specifically target SSM failure modes.

3.2 Spectral Koopman Attention

State-space models compress the entire history into a fixed-size recurrent state, which limits their ability to retrieve specific facts from long contexts [16?]. Spectral Koopman Attention (SKA) addresses this by estimating a compact linear dynamical model from prefix statistics and using it to retrieve values at query positions, without materializing a $T \times T$ attention matrix.

Projection. Given input $x \in \mathbb{R}^{B \times T \times d}$, learned linear projections produce keys, queries, and values per head:

$$z_t = W_k x_t \in \mathbb{R}^r, \quad z_t^q = W_q x_t \in \mathbb{R}^r, \quad v_t = W_o x_t \in \mathbb{R}^P, \quad (2)$$

where r is the rank of the Koopman operator (a hyperparameter controlling capacity vs. cost) and $P = d/H$ is the per-head dimension. Key and query projections are initialized orthogonally to promote well-conditioned covariance matrices at initialization.

Key normalization. To stabilize the covariance statistics, all keys and queries are divided by a shared normalization factor:

$$\hat{z}_t = \frac{z_t}{m}, \quad \hat{z}_t^q = \frac{z_t^q}{m}, \quad m = \max_{s \leq T} \|z_s\|_2, \quad (3)$$

clamped at $m \geq 10^{-6}$. This shared scaling ensures that keys and queries live on comparable scales without distorting their relative geometry, and prevents numerical overflow in the outer products that follow.

Chunk-causal accumulation. To preserve causality while enabling parallel computation, the sequence is partitioned into C chunks of size S (default $S = 64$). For each chunk c , three sufficient statistics are accumulated from all *preceding* chunks $0, \dots, c-1$ only (exclusive prefix sum), ensuring that the operator estimate at chunk c depends only on strictly earlier tokens:

$$G^{(c)} = \sum_{t \in \text{chunks } 0..c-1} \hat{z}_t \hat{z}_t^\top + \varepsilon I_r, \quad (4)$$

$$M^{(c)} = \sum_{\substack{t \in \text{chunks } 0..c-1 \\ t > 0}} \hat{z}_t \hat{z}_{t-1}^\top, \quad (5)$$

$$C_o^{(c)} = \sum_{t \in \text{chunks } 0..c-1} v_t \hat{z}_t^\top, \quad (6)$$

where ε (default 10^{-3}) is a ridge regularization parameter. The cross-covariance M includes *boundary transitions* between adjacent chunks: the transition $\hat{z}_{c,0} \hat{z}_{c-1,-1}^\top$ from the last token of chunk $c-1$ to the first token of chunk c is included in the statistics available to chunk c , since this transition is fully observed before chunk c begins processing. Note that C_o requires no boundary correction, as it captures per-token value-key associations rather than cross-token transitions.

The per-chunk statistics G_c , M_c , $C_{o,c}$ are computed in parallel via batched einsum operations, and the exclusive prefix sums are computed via cumsum with a one-position shift.

Koopman operator estimation. The Gram matrix $G^{(c)}$ is factored via batched Cholesky decomposition: $G^{(c)} = L L^\top$. When the initial factorization fails (detected via `cholesky_ex` info flags), a jittered version $G^{(c)} + 10^{-4}I$ is used as a fallback, avoiding CPU-GPU synchronization from a conditional branch. All covariance accumulation and linear algebra is performed in FP32 regardless of the training precision of the rest of the network.

The Koopman transition operator and value readout operator are:

$$A_w = M G^{-1}, \quad B_o = C_o G^{-1}, \quad (7)$$

computed via Cholesky solves $A_w^\top = L^{-\top} L^{-1} M^\top$ and $B_o^\top = L^{-\top} L^{-1} C_o^\top$.

Spectral normalization. To ensure training stability, A_w is normalized by its leading singular value via power iteration (6 iterations, detached from the computation graph):

$$A_w \leftarrow \frac{\gamma}{\max(\sigma_{\max}(A_w), 1)} A_w, \quad (8)$$

where γ is a learnable scalar clamped to $[1.0, 1.5]$. This bounds the spectral radius of A_w , preventing unbounded amplification in the power spectral filter while allowing gradients to flow through A_w itself (straight-through estimation).

Power spectral filter. Rather than computing an explicit eigen-decomposition, persistent dynamical modes are amplified relative to transient ones by repeated application of A_w . Given a whitened query $w_q = L^{-1}z^q$, the filtered output is:

$$w_f = A_w^K w_q, \quad (9)$$

where K (default 2) controls selectivity. Modes with eigenvalue magnitude $|\lambda_i|$ are amplified by $|\lambda_i|^K$, so transient modes ($|\lambda_i| \ll 1$) are exponentially suppressed. This is fully differentiable and avoids the numerical fragility of eigendecomposition in mixed-precision training.

Value retrieval. The filtered query is mapped back to the original coordinate frame and combined with the value readout:

$$z_f = L w_f, \quad \hat{y} = B_v z_f \in \mathbb{R}^P. \quad (10)$$

The output is scaled by a learnable parameter η (initialized to 1.5) and projected back to model dimension via a linear layer initialized to zero, ensuring that SKA begins training as a near-identity perturbation of the residual stream.

Triton kernel. When available, the post-Cholesky matmul chain (power filter $\rightarrow$ coordinate transform $\rightarrow$ value readout) is fused into a single Triton kernel that loads all four matrices (A_w , L , B_v , w_q) into SRAM and performs $K + 2$ matrix multiplications without HBM round-trips. Matrices are padded to the next power of two for t1.dot compatibility. This yields a measurable speedup for small r (where kernel launch overhead dominates) and falls back to batched PyTorch when Triton is unavailable.

3.3 Spectral Koopman MLP

Standard Transformer feedforward layers use a gated nonlinearity (SwiGLU [?]) with three weight matrices of size $d \times d_{ff}$, totaling $3 d d_{ff}$ parameters. We replace these with a *Spectral Koopman MLP* that implements the feedforward transformation as a single step of a spectrally-constrained linear dynamical system, requiring only two weight matrices ($2 d d_k$ parameters, a $\frac{1}{3}$ reduction).

Lift. The input is projected into a latent ‘‘Koopman’’ space via a single linear layer followed by SiLU activation:

$$g = \text{SiLU}(W_{\text{lift}} h) \in \mathbb{R}^{d_k}, \quad (11)$$

where $d_k = \lceil d \cdot e/64 \rceil \times 64$ with expansion factor $e \approx 8/3$ (matching the SwiGLU convention), and h is the layer-normed input.

Block-diagonal rotation. The lifted representation is reshaped into $d_k/2$ pairs $(g_1^{(i)}, g_2^{(i)})$ and evolved by a block-diagonal rotation matrix parameterized by learnable eigenvalues:

$$\begin{pmatrix} z_1^{(i)} \\ z_2^{(i)} \end{pmatrix} = \begin{pmatrix} \gamma_i & \omega_i \\ -\omega_i & \gamma_i \end{pmatrix} \begin{pmatrix} g_1^{(i)} \\ g_2^{(i)} \end{pmatrix}, \quad (12)$$

where (γ_i, ω_i) are the real and imaginary parts of a complex eigenvalue $\lambda_i = \gamma_i + i \omega_i$.

Spectral normalization of eigenvalues. To prevent unbounded growth through the rotation, the eigenvalue modulus is clamped to the unit disk:

$$|\lambda_i| = \sqrt{\gamma_i^2 + \omega_i^2}, \quad s_i = \frac{\min(|\lambda_i|, 1)}{|\lambda_i|}, \quad \gamma_i \leftarrow s_i \gamma_i, \quad \omega_i \leftarrow s_i \omega_i. \quad (13)$$

This is the pointwise analogue of the spectral normalization applied to the Koopman operator A_w in SKA: both enforce spectral radius ≤ 1 to guarantee stable dynamics.

Readout. The rotated pairs are interleaved back into a vector $z \in \mathbb{R}^{d_k}$ and projected to model dimension:

$$\text{output} = x + W_{\text{readout}} z. \quad (14)$$

Gated variant. An optional gated variant adds a sigmoid gate $\sigma(W_{\text{gate}} h)$ applied elementwise to z before the readout projection, at the cost of one additional $d \times d_k$ weight matrix. This brings the parameter count to parity with SwiGLU while retaining the spectral structure.

3.4 Architecture and Baselines

Full model (Mamba-2 + SKA + Koopman MLP). The complete architecture follows the Nemotron-H [?] hybrid layout: N blocks, each consisting of a sequence layer followed by a feedforward layer. The majority of sequence layers are Mamba-2; a fraction (25% in our 50M experiments) are replaced with SKA layers, evenly spaced throughout the network. All feedforward layers use the Spectral Koopman MLP. The first and last layers are always Mamba-2.

For a 50M-parameter model, we use $d = 448$, $N = 16$ layers, 4 SKA layers at positions $\{3, 7, 11, 15\}$, rank $r = 56$, $H = 7$ heads, chunk size $S = 64$, and $d_{\text{state}} = 64$ for Mamba-2. For a 180M-parameter model, we use $d = 768$, $N = 24$ layers, 2 SKA layers at positions $\{8, 16\}$, rank $r = 48$, and $H = 12$ heads.

Baseline: Mamba-3 + SKA (synthetic). For synthetic experiments that target specific retrieval failure modes, we use Mamba-3 as the SSM backbone with SKA modules inserted at the final layers. This isolates SKA’s retrieval contribution in a controlled setting with minimal confounds from language modeling objectives.

3.5 O(1) Recurrent Inference

A key property of this architecture for agentic deployment is that every component admits $O(1)$ state during autoregressive generation, constant in sequence length.

- **Mamba-2/3:** maintains a convolutional state of size $d_{\text{inner}} \times d_{\text{conv}}$ and an SSM state of size $n_{\text{heads}} \times d_{\text{head}} \times d_{\text{state}}$, both fixed regardless of T .
- **SKA:** maintains running accumulators $G \in \mathbb{R}^{r \times r}$, $M \in \mathbb{R}^{r \times r}$, $C_v \in \mathbb{R}^{P \times r}$, the previous key $z_{t-1} \in \mathbb{R}^r$, and a scalar max norm—totaling $2r^2 + Pr + r + 1$ floats per head, independent of T . At each step, the accumulators are updated as $G \leftarrow G + \hat{z}_t \hat{z}_t^T$, $M \leftarrow M + \hat{z}_t \hat{z}_{t-1}^T$, $C_v \leftarrow C_v + v_t \hat{z}_t^T$, and the Koopman operator is re-estimated from the updated statistics.
- **Koopman MLP:** stateless (pointwise).

For the 50M model ($r = 56$, $P = 64$, $H = 7$, 4 SKA layers), the total SKA state is approximately 77 KB in FP32. Combined with Mamba-2 state, the total recurrent state per layer is on the order of

hundreds of kilobytes—enabling generation over arbitrarily long contexts with bounded memory.

This property is critical for agentic systems where a model must process long sequences of tool calls, retrieved documents, and multi-turn dialogue without memory growing linearly in context length.

3.6 Complexity Summary

Table 1: Per-layer complexity comparison. T : sequence length, d : model dimension, r : SKA rank, S : chunk size.

Component	Train (parallel)	Infer (per step)
Causal Attention	$O(T^2d)$	$O(Td)$
Mamba-2	$O(Td)$	$O(d)$
SKA	$O(Tr^2/S + r^3)$	$O(r^3)$
SwiGLU MLP	$O(Td d_{\text{ff}})$	$O(d d_{\text{ff}})$
Koopman MLP	$O(Td d_k)$	$O(d d_k)$

During training, SKA’s cost is dominated by the $O(r^3)$ Cholesky factorization per chunk per head, amortized over S tokens. Since $r \ll T$, the total cost is sublinear in T . During inference, each step requires an $O(r^3)$ solve (re-estimating the operator from updated accumulators), which is negligible for $r \leq 64$. The Koopman MLP saves $\frac{1}{3}$ of the parameters relative to SwiGLU at identical FLOPs per token.

4 Experimental Setup

We evaluate across two experimental regimes: controlled synthetic tasks that isolate retrieval failure modes of pure SSMs (§4.1), and from-scratch language modeling that tests whether SKA and Koopman MLPs scale to realistic pretraining (§4.2). Both regimes are motivated by failure modes observed in agentic systems, where an orchestration loop must recover earlier tool outputs or configuration values after processing long intermediate reasoning traces.

4.1 Synthetic Tasks

4.1.1 Baselines and Parameter Budget. A natural objection to any hybrid architecture is that the improvement may stem from additional parameters rather than architectural mechanism. We address this with two baseline strategies.

For Tool Trace, we use an asymmetric capacity control: the two SKA variants share a base configuration of $d=96$, $H=3$ heads, and $L=3$ layers (~414K parameters), while the Mamba-3 baseline is deliberately scaled to $d=128$, $H=4$, $L=4$ (~863K parameters, approximately $2\times$ the SKA count). If Mamba-3 at double the parameter budget still exhibits a memory cliff where SKA does not, the gap cannot be attributed to capacity alone. The three models compared in Tool Trace are:

- (1) **Mamba-3 ($2\times$)**: SSM-only baseline at $\sim 2\times$ the parameter budget of the SKA variants.
- (2) **SKA ($\eta=1.5$)**: Mamba-3 augmented with Spectral Koopman Attention at moderate retrieval strength.
- (3) **SKA ($\eta=2.5$)**: same architecture with boosted retrieval scale, testing the dose–response relationship of the SKA signal.

For the CoT Retrieval experiments, all models share the same base dimensions ($d=96$, $H=3$, $L=3$, $V=128$) and we compare three variants: Mamba-3, SKA ($\eta=1.5$), and SKA ($\eta=2.5$).

Shared configuration. All SSM variants use chunked SSD recurrence (chunk size 64) with the scan path in FP32. All SKA variants use rank $r=24$, ridge $\epsilon=10^{-3}$, power filter order $K=2$, spectral normalization with SSN $\gamma \in [1.0, 1.5]$, and sequence-max normalization. Training uses AdamW ($\beta_1=0.9$, $\beta_2=0.95$, weight decay 0.01), gradient clipping at 1.0, learning rate 3×10^{-4} with cosine decay.

4.1.2 Tool Trace Delayed Commit. Tool Trace is designed to stress-test long-range associative retrieval with high-entropy answers and out-of-distribution gap lengths.

Task. Each episode consists of three regions: a fact table, a distractor gap, and a query. The fact table contains $n \in [4, 8]$ bindings of the form agent: tool→result, where each result is a random 4-character base-36 string drawn uniformly from $36^4 \approx 1.68 \times 10^6$ possibilities. The distractor gap is filled with syntactically varied but informationally empty log entries that reference the same agent and tool names without revealing any bindings. The query names one agent–tool pair and the model must produce the corresponding 4-character result token by token. Random guessing achieves approximately $1/36 \approx 2.8\%$ per character, making memorization of surface statistics ineffective. All text is encoded at byte level ($V=256$).

Training tiers. We train on two distractor-length ranges to study both in-distribution accuracy and length generalization:

- **Short** (128–512 tokens): the distractor gap is comparable to the fact table, so even a decaying SSM state may retain enough signal.
- **Medium** (512–1,536 tokens): the gap exceeds the fact table by up to $10\times$, requiring robust long-range retrieval.

Both tiers use 4–8 bindings, 6 agents, and are trained for 15,000 steps with batch size 16.

Length generalization sweep. After training, each model is evaluated on fixed distractor gap lengths of 512, 1,024, 2,048, 4,096, and 8,192 tokens (200 episodes each). For the Short tier this tests gaps up to $16\times$ longer than training; for Medium, up to $\sim 5\times$. This sweep tests whether SKA’s retrieval mechanism generalizes beyond the training distribution, or whether accuracy degrades gracefully compared to the sharp cliff exhibited by recurrence-only models.

SKA diagnostics. At each evaluation checkpoint we log three quantities for the learned Koopman operator A_w : the spectral radius before normalization (σ_{pre}), which tracks how aggressively spectral normalization must intervene; the maximum eigenvalue magnitude after normalization ($|\lambda|_{\text{max}}$), which should remain near but below 1.0 for a stable operator; and the fraction of eigenvalues exceeding the persistence threshold ($|\lambda| > 0.5$), which measures how many modes the operator retains as long-term memory channels.

4.1.3 System Prompt CoT Retrieval. This experiment targets a failure mode common in deployed agentic systems: critical configuration variables are set in a system prompt, the model then processes a long chain-of-thought scratchpad, and a subsequent query demands exact recall of a value from the original prompt.

Sub-task A: CoT Retrieval. A system prompt defines $n=4$ named variables with two-digit values, e.g. `SYS|A: 42|B: 99|C: 15|D: 88|`. This is followed by a CoT scratchpad of bracketed pseudo-actions (e.g. `[scan_X][wait_Y][ping_Z]...`) that are syntactically structured but carry no information about the system variables. A query then asks for one variable’s value. The scratchpad is pure noise: it does not reuse the variable names, so the model need only ignore it.

Sub-task B: Specific Recall. A harder variant where $n=6$ facts are set with `SET:A=42|SET:B=17|...`, followed by distractor lines of the form `LOG:A.ok|LOG:C.err|...` that reference the *same entity names* in irrelevant contexts. This tests whether the model can distinguish a binding (`SET:A=42`) from a mention (`LOG:A.ok`), a discrimination that requires content-aware retrieval rather than simple pattern matching on entity names.

Tiers. Each sub-task is evaluated at two difficulty levels:

- **Easy:** 10–30 CoT or distractor steps (~100–300 token gap).
- **Hard:** 150–300 CoT steps for sub-task A (~1,200–2,400 token gap); 80–200 distractor steps for sub-task B.

All tiers are trained for 2,500 steps with batch size 16.

Expected separation. On the Easy tiers, the gap is short enough that even an exponentially decaying SSM state retains sufficient signal, and all models should perform comparably. On the Hard tiers, the SSM state has been overwritten by hundreds of scratchpad tokens; SKA’s Koopman snapshot should recover the original bindings by identifying them as persistent modes and suppressing the transient scratchpad through the power filter.

4.2 Language Modeling

The synthetic experiments demonstrate SKA’s retrieval advantage in isolation. The language modeling experiments test whether this advantage persists when SKA and Koopman MLPs are integrated into a full-scale pretraining pipeline, where the model must simultaneously learn syntax, semantics, world knowledge, and long-range retrieval.

4.2.1 Models. We train two matched 50M-parameter models from scratch:

- (1) **Mamba-2 + SKA + Koopman MLP** (our method): 16 layers with $d=448$, 4 SKA layers at positions {3, 7, 11, 15} (25% of layers), rank $r=56$, $H=7$ heads, chunk size $S=64$, $d_{\text{state}}=64$ for Mamba-2, and Spectral Koopman MLP with expansion factor $8/3$ at every layer. SKA accounts for approximately 6% of total parameters; Koopman MLPs account for approximately 35%. Total: ~50M parameters.
- (2) **Mamba-2 + Attention + SwiGLU** (baseline): Matched architecture with the same layer count ($N=16$), model dimension ($d=448$), and head configuration ($H=7$), but with standard multi-head causal softmax attention replacing SKA and SwiGLU feedforward layers replacing Koopman MLPs. Attention layers are placed at the same positions {3, 7, 11, 15} to control for layer placement effects. Total: ~50M parameters.

Both models use the Mistral-7B tokenizer (vocabulary size 32,000) and tied input–output embeddings.

4.2.2 Training Configuration.

Data. Both models are trained on FineWeb-Edu [?] (sample-10BT split), a curated subset of Common Crawl filtered for educational content quality. Text is tokenized and packed into fixed-length chunks of 2,048 tokens with no padding; an end-of-sequence token is inserted between documents.

Optimization. Training uses DeepSpeed ZeRO Stage 1 with BF16 mixed precision. The optimizer is AdamW with $\beta_1=0.9$, $\beta_2=0.95$, weight decay 0.1, and gradient clipping at 1.0. The learning rate follows a cosine schedule with peak 6×10^{-4} , 2,000 warmup steps, and minimum LR ratio 0.1. We train for 50,000 optimizer steps with an effective batch size of 96 sequences ($96 \times 2,048 \approx 197\text{K}$ tokens per step, ~9.8B tokens total).

Hardware. All language model training is performed on a single NVIDIA B200 GPU (183 GB HBM3e).

4.2.3 Evaluation. We evaluate both models on the following benchmarks after training is complete.

Held-out perplexity. WikiText-103 [?] test split, packed into 2,048-token chunks. This measures general language modeling quality on unseen data with zero overlap with the FineWeb-Edu training set.

Standard LM benchmarks. We report results on the lm-evaluation-harness [?] suite: HellaSwag (commonsense reasoning), PIQA (physical intuition), ARC-Easy and ARC-Challenge (science QA), WinoGrande (coreference resolution), and LAMBADA (long-range cloze prediction). These benchmarks test whether the architectural changes degrade general language understanding relative to the attention baseline.

Needle-in-a-Haystack (NIAH). We evaluate factual recall at context lengths 128, 256, 512, 1,024, 2,048, and 4,096 tokens using three difficulty levels:

- **NIAH-Single-1:** A single fact (“The special number is NNNN.”) is inserted at a random depth in a haystack of filler sentences. The model must answer “What is the special number?”
- **NIAH-Single-2:** Same as Single-1 but the haystack contains distractor facts with similar format (“The *regular/normal/common* number is MMMM.”), requiring the model to distinguish the target binding from similar-looking alternatives.
- **NIAH-Single-3:** The fact is inserted as in Single-1 but the query is paraphrased (“Recall the unique designated number”), testing semantic matching rather than exact key recall.

Each condition uses 200 examples with greedy decoding via the $O(1)$ recurrent inference wrapper (§3.5). NIAH is the primary evaluation for the agentic use case: it directly measures whether the model can retrieve a specific fact from a long context, the core operation underlying tool output retrieval, system prompt recall, and multi-turn conversation grounding.

Memory profiling. We measure peak GPU memory during autoregressive generation at increasing context lengths (512 to 8,192 tokens) to empirically verify the $O(1)$ inference state property. The

recurrent wrapper prefills the prompt via the parallel forward pass, then generates tokens one at a time using fixed-size state. We report both prefill memory (expected $O(T)$) and per-step memory (expected $O(1)$) separately. This evaluation is particularly relevant for agentic deployment, where models must generate over long accumulated contexts (tool call histories, retrieved documents, multi-turn dialogue) without memory scaling linearly in context length.

5 Results

5.1 Tool Trace Delayed Commit

Figure ?? reports in-distribution training curves for the short (128–512 token) and medium (512–1,536 token) distractor regimes. Mamba-3 without SKA never rises above chance-level accuracy (2–8%) on either tier, consistent with the exponential memory decay documented by [29]. Scaling the bare SSM to 2.5M parameters did not change this outcome: the recurrent state simply cannot retain high-entropy bindings across distractor gaps of this length. In contrast, both CG+SKA variants learn the task, with the $\eta = 2.5$ configuration reaching approximately 50–60% evaluation accuracy on the short tier and comparable performance on the medium tier within 6,000 training steps. The $\eta = 1.5$ variant follows a similar trajectory but plateaus slightly lower, suggesting that the SKA output scaling interacts meaningfully with the magnitude of the residual stream. The more striking result is length generalization. Figure ?? evaluates all models on distractor gaps of 512 to 8,192 tokens—up to 16x longer than the longest gaps seen during training. Mamba-3 remains at chance across all evaluation lengths. The CG+SKA models, however, maintain high accuracy well beyond their training distribution: on the short-trained models, both η configurations sustain accuracy above 80% at 8,192-token gaps, and the medium-trained variants exhibit a similarly flat accuracy profile across the entire sweep. This is the expected behavior of a retrieval mechanism whose sufficient statistics are sequence-length-independent: because SKA accumulates fixed-size covariance matrices rather than a growing cache, there is no architectural reason for accuracy to degrade with context length once the operator has learned to identify the relevant bindings. The absence of any visible accuracy cliff in the CG+SKA curves, compared with the flat-at-chance Mamba-3 baseline, confirms that the spectral Koopman operator, not the SSM recurrence, is responsible for the retrieval capability.

The action mask (??) provides an inductive bias by restricting covariance accumulation to fact-bearing tokens. However, an ablation in which the action mask is set to $m_t = 1$ for all tokens shows no meaningful accuracy degradation (Appendix, Figure 1), confirming that the spectral filter successfully separates persistent fact modes from transient distractor modes without explicit token-level supervision.

5.2 System Prompt CoT Retrieval

Table 2 reports accuracy on the System Prompt CoT Retrieval task. On both the Easy tier (10–30 CoT steps) and the Hard tier (150–300 CoT steps), both CG+SKA configurations achieve perfect retrieval accuracy, while Mamba-3 remains near chance at 13.3% and 16.0% respectively. The near-chance baseline is consistent with the memory cliff hypothesis: even on the Easy tier, where the distractor gap is only 100–300 tokens, the SSM’s recurrent state has already lost

the system-prompt bindings. That accuracy does not meaningfully differ between Easy and Hard for Mamba-3 suggests that the failure is not gradual decay but a rapid loss of high-entropy content once any substantial distractor span intervenes.

The training dynamics (Figure 2, Appendix H.2) reveal that both CG+SKA variants converge to perfect accuracy within approximately 500 training steps on the Easy tier and within 1,000 steps on the Hard tier, while Mamba-3 shows no upward trend throughout training. The convergence is notably faster than on Tool Trace, likely because the CoT task involves fewer bindings ($n=4$ variables versus 4–8 tool results) and the answer is a two-digit number rather than a four-character base-36 string, reducing the per-query entropy. The identical performance of the $\eta=1.5$ and $\eta=2.5$ variants on this task indicates that once the SKA signal is strong enough to override the decayed recurrent state, additional scaling provides no further benefit—the task is effectively saturated.

Table 2: System Prompt CoT Retrieval accuracy. Mamba-3 fails to recover system-prompt variables after a chain-of-thought scratchpad on both difficulty tiers, while CG+SKA achieves perfect accuracy regardless of scratchpad length.

Model	CoT Easy	CoT Hard
Mamba-3	13.3%	16.0%
CG+SKA ($\eta = 1.5$)	100%	100%
CG+SKA ($\eta = 2.5$)	100%	100%

6 Discussion

The SKA module involves a number of interacting design choices, several of which emerged from diagnosing failures in earlier iterations. This section explains the rationale behind each choice, connects it to the theory, and identifies limitations and open questions.

6.1 Why Rotation-Invariant Features?

The core SSM in Mamba-3 applies rotary position encoding (RoPE) to its key and query vectors before they enter the recurrence. After t steps, a key vector k_t has accumulated a position-dependent phase $\theta_t = \sum_{s=1}^t \delta_s \omega$, so that the rotated key lies on the orbit of $\text{SO}(2)^{N/2}$ acting on $\mathbb{R}^N$. Any module that computes pairwise statistics over these features, as SKA does when building the Gram matrix G and cross-covariance M , must contend with the fact that two tokens encoding the same content at different positions will have different rotated representations.

This invariance is what allows the Gram matrix $G = \sum_t z_t z_t^T$ to function as a content-addressed index rather than a position-dependent correlation matrix. Without it, the Koopman operator A_w would need to simultaneously learn the dynamics of interest *and* undo the position encoding, a conflation that empirically prevents convergence of the spectral structure.

6.2 Independent Projections vs. Legacy CG Path

The theoretical motivation above assumes SKA operates on Mamba’s rotary features directly (the *legacy* path). In practice, all experiments

in this paper use the *independent* path, where SKA receives its own learned projections from the residual stream:

$$z_t = W_K h_t, \quad z_t^{(q)} = W_Q h_t, \quad v_t^{\text{ska}} = W_V h_t, \quad (15)$$

where h_t is the post-Mamba residual after layer normalization, and $W_K, W_Q \in \mathbb{R}^{Hr \times d}$, $W_V \in \mathbb{R}^{HP \times d}$ are initialized with orthogonal weights.

This decoupling is motivated by a practical observation: Mamba’s key and value features are optimized for the SSM recurrence, which imposes conflicting demands on the feature space. The recurrence needs features that produce smooth, slowly varying state trajectories (favouring low-rank, temporally correlated representations), while SKA needs features that produce *discriminative* inner products for content addressing (favouring high-rank, semantically clustered representations). Sharing features forces a compromise that degrades both.

The independent path achieves position invariance implicitly: because the residual stream h_t has already been processed by layer normalization and does not carry explicit rotary phase, the learned projections W_K and W_Q can directly encode content-based similarity without needing an explicit CG lift. The theoretical role of the CG invariants, ensuring that G indexes content rather than position, is thus preserved, but the mechanism shifts from an analytic invariant map to a learned projection that is free to discover whatever invariant structure the task requires.

We retain the name “CG+SKA” to indicate that the invariant lift is the *design principle* motivating the feature architecture, even though the independent path implements it via learned projections rather than explicit polynomial invariants.

6.3 Whitened Koopman Operator

The raw cross-covariance $M = \sum_t z_{t+1} z_t^\top$ and Gram matrix $G = \sum_t z_t z_t^\top$ have magnitudes that depend on the norm and count of feature vectors, not on the dynamics of interest. Whitening via $A_w = L^{-1} M L^{-\top}$, where L is the Cholesky factor of the regularized Gram $\tilde{G} = G + \epsilon I$, removes this scale dependence and ensures that the eigenvalues of A_w are interpretable as dynamical quantities: $|\lambda| \approx 1$ corresponds to a mode that persists across timesteps, while $|\lambda| \ll 1$ corresponds to a transient mode.

This interpretation is what makes the power filter and spectral diagnostics meaningful. Without whitening, the largest eigenvalue of M would reflect feature scale rather than persistence, and the spectral filter would suppress or amplify modes based on magnitude rather than dynamical relevance.

A subtle implementation detail: the whitening must be computed as $L^{-1} M L^{-\top}$ (solving two triangular systems) rather than the naïve $\tilde{G}^{-1} M$, which is not symmetric and produces a different eigenspectrum. An earlier version of our code used the incorrect form, which prevented the spectral structure from emerging.

6.4 Spectral Normalization and Variance Restoration

The whitened operator A_w can have spectral radius $\sigma_{\max} \gg 1$ early in training, before the feature projections have converged. Applying the power filter A_w^K to such an operator amplifies the

leading eigenvalue exponentially, producing gradient spikes and numerical overflow.

We constrain the operator via spectral normalization: $\hat{A}_w = A_w / \max(1, \hat{\sigma}_{\max})$, where $\hat{\sigma}_{\max}$ is estimated by 6 steps of power iteration with deterministic initialization. This bounds eigenvalues ($|\lambda| \leq 1$) after normalization, ensuring contractive power filter.

However, normalization introduces a variance deficit: if all eigenvalues are scaled down by the same factor, modes near $|\lambda| = 1$ (the persistent modes we want to preserve) may be pushed below the filter threshold. We restore variance with a learned scalar $\gamma \in [1.0, 1.5]$, applied as $A_w \leftarrow \gamma \hat{A}_w$. Clamping γ to $[1.0, 1.5]$ allows partial restoration without re-introducing instability. In practice, γ converges to values near 1.2–1.4, indicating that the operator benefits from operating slightly above the strict unit-spectral-radius boundary.

6.5 Power Filter: Separating Persistent from Transient Modes

The power filter computes $A_w^K z_q$ for each query feature z_q , raising eigenvalues to the K -th power. At $K=2$, a mode at $|\lambda| = 0.9$ retains 81% energy, while a mode at $|\lambda| = 0.3$ retains only 9%. This soft threshold preserves persistent modes (stored bindings) and suppresses transients (distractor noise), bypassing explicit mode classification.

The choice $K=2$ is deliberately conservative. An earlier version used $K=4$, which sharpened the separation but crushed everything below $|\lambda| \approx 0.85$ to near-zero, leaving too few active modes for the value readout B_v to reconstruct the answer. $K=2$ preserves a wider band of moderately persistent modes, which carry partial information about related bindings that can contribute to disambiguation.

We also implemented a Schur spectral filter that computes the real Schur decomposition of A_w , applies a soft threshold to eigenvalue magnitudes, and reconstructs a filtered operator. This is mathematically cleaner but requires `torch.linalg.schur`, which is not available on all platforms. Where available, the Schur filter produces comparable results to the power filter at $K=2$.

6.6 Why Ungated Additive Injection?

An early design used a sigmoid gate to blend SKA’s output with the Mamba residual: $h \leftarrow (1-g) \cdot h + g \cdot h_{\text{ska}}$, where $g = \sigma(W_g h + b_g)$ and the gate bias was initialized at -2.0 so that $\sigma(-2) \approx 0.12$. Combined with a retrieval scale of $\eta = 0.5$, this meant SKA’s effective influence on the residual stream was approximately $0.12 \times 0.5 = 6\%$, far too small to shift the logits at answer positions.

The fundamental problem is a gradient bottleneck: the loss is computed only at 2-4 answer tokens per sequence, and the gate must learn to open at exactly those positions. With such sparse supervision, the gate parameters receive negligible gradient and remain near their initialization indefinitely. This is a general failure mode of learned gates in settings where the supervised signal is extremely sparse relative to the sequence length.

The fix is to remove the gate entirely and use a direct additive connection $h \leftarrow h + \eta \cdot h_{\text{ska}}$ with a boosted scale $\eta \in \{1.5, 2.5\}$. This ensures the SKA signal has sufficient magnitude to influence predictions from the start of training, allowing the feature projections

and Koopman operator to receive meaningful gradient. The dose-response comparison between $\eta = 1.5$ and $\eta = 2.5$ tests whether more SKA signal monotonically improves retrieval or eventually overwhelms Mamba’s local context processing.

6.7 Sequence-Max Normalization

The rank- r features z_t can vary in norm across the sequence, particularly between fact-table tokens (which encode structured bindings) and distractor tokens (which encode repetitive noise patterns). If the Gram matrix G is dominated by a few high-norm tokens, the Koopman operator will overfit to those tokens’ dynamics and ignore the rest of the fact table.

Sequence-max normalization divides features in a sequence’s maximum feature norm observed in that sequence: $z_t \leftarrow z_t / \max_s \|z_s\|$. This bounds the entries of G without amplifying noise (unlike ℓ_2 -normalization per token, which would inflate low-norm distractor features to unit norm). The same normalization is applied to query features z_q keeping the query in the same scale as the keys.

6.8 Ridge Regularization and Numerical Stability

The regularized Gram $\tilde{G} = G + \varepsilon I$ serves two purposes. First, it ensures $\tilde{G}$ is positive definite and hence admits a Cholesky factorization, even when the number of tokens is less than the rank r or when features are nearly collinear. Second, the ridge $\varepsilon = 10^{-3}$ acts as a prior that shrinks the Koopman operator toward zero in directions with little data support, preventing the operator from hallucinating dynamics in low-variance subspaces.

Despite the ridge, Cholesky factorization can still fail in low-precision arithmetic or with adversarial feature configurations. We use a robust Cholesky routine that symmetrizes the input, retries with geometrically increasing jitter (10^{-4} , 10^{-3} , 10^{-2} , 10^{-1}), and falls back to an eigendecomposition with clamped eigenvalues as a last resort. All Koopman computations (Gram matrix, cross-covariance, Cholesky, triangular solves, power iteration, and the power filter) are performed in FP32 regardless of the training precision, as we found that FP16 accumulation introduces sufficient error to corrupt the eigenspectrum.

6.9 The Action Mask: Provenance-Aware Operator Estimation

When the Gram matrix G and cross-covariance M are estimated from the full prefix, distractor tokens contribute noise to both matrices. If the distractor gap is long relative to the fact table, the operator A_w is estimated primarily from distractor dynamics, which slow training toward retrieval.

The action mask restricts the summation to fact-table tokens only: $G = \sum_{t:m_t=1} z_t z_t^\top$. This ensures the Koopman operator captures the dynamics of the informative region, even when it constitutes a small fraction of the prefix.

In agentic deployments, such provenance information is typically available without additional annotation: tool-call responses, API results, and system-prompt injections are already demarcated by the orchestration framework. The action mask simply propagates this existing structure into the operator estimation. Importantly, the mask is not required for SKA to function. When no action mask

is provided, SKA defaults to using the full prefix mask, and the operator is estimated from all available tokens. The mask improves operator quality in high-noise settings but is not a prerequisite for the retrieval mechanism.

6.10 Limitations

Computational cost. SKA introduces $O(Tr^2)$ overhead per layer for the Gram matrix, Cholesky factorization, and power filter, plus $O(r^3)$ for the whitening and spectral normalization. At the ranks used in our experiments ($r = 24$), this is modest relative to the SSM recurrence, but it scales quadratically in rank and may become a bottleneck for very large r .

Batch-level operator. The current implementation estimates one Koopman operator per sequence. Sharing operator statistics across a batch or across layers could improve sample efficiency, but it would also risk conflating dynamics from unrelated sequences.

Synthetic benchmarks. Both benchmarks are synthetic, with clean separations between fact regions, distractor regions, and queries. Natural language does not have such clean structure, and it remains an open question whether the memory cliff phenomenon and SKA’s recovery generalize to realistic agentic workloads with interleaved tool calls, partial observations, and ambiguous context boundaries.

Scale. Our models are small ($\sim 400K$ – $860K$ parameters) and our sequences are moderate by modern standards. Our larger scale experiments use Mamba 2’s 130M parameter checkpoint with LoRA Mamba 2 fine tuned for retrieval, compared with Mamba 2 + attention fine tuned for retrieval, vs Mamba 2 + SCA fine tuned for retrieval. ($\leq 8,192$ tokens at evaluation). We view this as a strength of the approach: the fact that SKA produces clear, measurable improvements at sub-million parameter scale suggests that the mechanism is not relying on brute-force memorization but is capturing genuine structural properties of the retrieval problem. Small-model expressivity also makes the method practical for on-device and latency-sensitive agentic deployments where billion-parameter models are infeasible. The $O(Tr^2)$ complexity of the SKA module suggests that scaling to longer contexts and larger models should remain tractable, though validating this is left to future work.

7 Conclusion

We identified the memory cliff as a fundamental failure mode of state-space models: beyond a task-dependent horizon, retrieval accuracy collapses to chance because the fixed-size recurrent state cannot preserve high-entropy bindings against the compressive pressure of intervening tokens.

We introduced Spectral Koopman Attention (SKA), a module that reframes content-addressed retrieval as kernel ridge regression over streaming sufficient statistics. By accumulating key–value covariance matrices of fixed size $O(r^2 + rP)$, fitting a whitened Koopman operator, and retrieving through a power-iterated spectral filter, SKA provides SSM layers with an explicit long-range retrieval pathway that requires no KV cache and no attention layers. The Clebsch–Gordan invariant lift, or its learned equivalent via independent projections, ensures that the accumulated statistics index content rather than position, resolving the nonstationarity introduced by rotary embeddings.

On two synthetic benchmarks motivated by agentic failure modes, Tool Trace associative recall and System Prompt CoT Retrieval, SKA-augmented models achieved near-perfect accuracy in regimes where bare Mamba-3 fails to exceed chance. The most striking result is length generalization: models trained on distractor gaps of at most 1,536 tokens maintained high accuracy at 8,192-token evaluation gaps with no visible degradation, confirming that the retrieval mechanism’s constant-memory statistics are genuinely sequence-length-independent. Ablations showed that the spectral operator, not the action mask, drives the retrieval gain, and that design choices including whitening order, ungated injection, and sequence-max normalization were each necessary for the mechanism to converge. Several limitations temper these conclusions. Our experiments operate at sub-million parameter scale on synthetic tasks with clean structural boundaries between facts, distractors, and queries. Natural-language agentic workloads present messier provenance signals, partial observations, and ambiguous context boundaries that may challenge the operator’s ability to separate persistent from transient modes without explicit masking. Scaling behavior at billions of parameters and contexts beyond 10^4 tokens remains unvalidated. The $O(Tr^2)$ per-layer overhead is modest at the ranks used here but could become a bottleneck at larger projection dimensions.

Despite these caveats, the core result is encouraging: a principled dynamical-systems mechanism can endow SSMs with exact associative retrieval in constant memory, closing the retrieval gap with attention without inheriting its linear memory cost. We believe this points toward a class of hybrid architectures in which recurrence handles local context and streaming dynamics while lightweight operator modules handle the sparse, long-range lookups that agentic workflows demand—all within a fixed memory envelope suitable for on-device and latency-sensitive deployment.

References

- [1] Muhammad Adnan, Akhil Arunkumar, Gaurav Jain, Prashant J. Nair, Ilya Solovey-chik, and Purushotham Kamath. 2024. Keyformer: KV Cache Reduction through Key Tokens Selection for Efficient Generative Inference. *Machine Learning and Systems (MLSys)* (2024). arXiv:2403.09054.
- [2] Anonymous. 2024. MiniCache: KV Cache Compression in Depth Dimension for Large Language Models. *arXiv preprint* (2024).
- [3] Anonymous. 2025. ChunkKV: Semantic-Preserving KV Cache Compression for Efficient Long-Context LLM Inference. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [4] Anonymous. 2025. Inference-Time Hyper-Scaling with KV Cache Compression. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- [5] Anonymous. 2025. Mechanistic Analysis of Associative Recall in State Space Models. *arXiv preprint arXiv:2505.15105* (2025).
- [6] Simran Arora, Sabri Eyuboglu, Aman Timalsina, Isys Johnson, Michael Poli, James Zou, Atri Rudra, and Christopher Ré. 2023. Zoology: Measuring and Improving Recall in Efficient Language Models. *arXiv preprint arXiv:2312.04927* (2023).
- [7] Simran Arora, Sabri Eyuboglu, Michael Zhang, Aman Timalsina, Silas Alberti, Dylan Zinsley, James Zou, Atri Rudra, and Christopher Ré. 2024. Simple Linear Attention Language Models Balance the Recall-Throughput Tradeoff. *arXiv preprint arXiv:2402.18668* (2024).
- [8] Steven L Brunton, Marko Budišić, Erika Kaiser, and J Nathan Kutz. 2021. Modern Koopman theory for dynamical systems. *arXiv preprint arXiv:2102.12086* (2021).
- [9] Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Baobao Chang, Junjie Hu, et al. 2024. PyramidKV: Dynamic KV Cache Compression Based on Pyramidal Information Funneling. *arXiv preprint arXiv:2406.02069* (2024).
- [10] Tri Dao and Albert Gu. 2024. Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality. In *International Conference on Machine Learning (ICML)*. arXiv:2405.21060.
- [11] Albert Gu et al. 2025. Mamba-3: Improved Sequence Modeling using State Space Principles. *ICLR 2026 Conference Submission* (2025). OpenReview:HwCvaJOiCj.
- [12] Albert Gu and Tri Dao. 2024. Mamba: Linear-Time Sequence Modeling with Selective State Spaces. *Conference on Language Modeling (COLM)* (2024). arXiv:2312.00752.
- [13] Albert Gu, Karan Goel, and Christopher Ré. 2022. Efficiently Modeling Long Sequences with Structured State Spaces. In *International Conference on Learning Representations (ICLR)*. arXiv:2111.00396.
- [14] Ankita Gupta, Albert Gu, and Jonathan Berant. 2022. Diagonal State Spaces are as Effective as Structured State Spaces. *Advances in Neural Information Processing Systems (NeurIPS)* 35 (2022).
- [15] HAX. 2025. Overcoming Long-Context Limitations of State-Space Models via Context-Dependent Sparse Attention. *arXiv preprint arXiv:2507.00449* (2025).
- [16] Samy Jelassi, David Brandfonbrener, Sham M. Kakade, and Eran Malach. 2024. Repeat After Me: Transformers are Better Than State Space Models at Copying. *arXiv preprint arXiv:2402.01032* (2024).
- [17] Bernard O Koopman. 1931. Hamiltonian systems and transformation in Hilbert space. *Proceedings of the National Academy of Sciences* 17, 5 (1931), 315–318.
- [18] Yuhong Li et al. 2024. SnapKV: Compressing KV Caches by Identifying Important Attention Features. *arXiv preprint* (2024).
- [19] Opher Lieber, Barak Lenz, et al. 2025. Jamba: A Hybrid Transformer-Mamba Language Model. In *International Conference on Learning Representations (ICLR)*. arXiv:2403.19887.
- [20] Zirui Liu et al. 2023. KIVI: A Tuning-Free Asymmetric 2bit Quantization for KV Cache. *arXiv preprint* (2023).
- [21] Bethany Lusch, J Nathan Kutz, and Steven L Brunton. 2018. Deep learning for universal linear embeddings of nonlinear dynamics. *Nature communications* 9, 1 (2018), 4950.
- [22] Igor Mezić. 2005. Spectral properties of dynamical systems, model reduction and decompositions. *Nonlinear Dynamics* 41, 1 (2005), 309–325.
- [23] NVIDIA. 2024. Hymba: A Hybrid-head Architecture for Small Language Models. *arXiv preprint* (2024).
- [24] NVIDIA. 2025. Nemotron 3: Efficient and Open Intelligence. *arXiv preprint arXiv:2512.20856* (2025).
- [25] NVIDIA. 2025. Nemotron-H: A Family of Hybrid Mamba-Transformer Models. *Technical Report* (2025).
- [26] Jimmy T.H. Smith, Andrew Warrington, and Scott W. Linderman. 2023. Simplified State Space Layers for Sequence Modeling. *International Conference on Learning Representations (ICLR)* (2023).
- [27] Konrad Staniszewski and Adrian Łańcucki. 2025. KV Cache Transform Coding for Compact Storage in LLM Inference. *arXiv preprint arXiv:2511.01815* (2025).
- [28] Roger Waleffe et al. 2024. An Empirical Study of Mamba-based Language Models. *arXiv preprint arXiv:2406.07887* (2024).
- [29] Peihao Wang, Ruisi Cai, Yuehao Wang, Jiajun Zhu, Pragya Srivastava, Zhangyang Wang, and Pan Li. 2024. Understanding and mitigating bottlenecks of state space models through the lens of recency and over-smoothing. *arXiv preprint arXiv:2501.00658* (2024).
- [30] Matthew O Williams, Ioannis G Kevrekidis, and Clarence W Rowley. 2015. A data-driven approximation of the koopman operator: Extending dynamic mode decomposition. *Journal of Nonlinear Science* 25, 6 (2015), 1307–1346.
- [31] Dongjie Yang, Xiaodong Han, Yan Gao, Yao Hu, Shilin Zhang, and Hai Zhao. 2024. PyramidInfer: Pyramid KV Cache Compression for High-throughput LLM Inference. In *Findings of the Association for Computational Linguistics: ACL 2024*. 3258–3270.
- [32] Songlin Yang et al. 2024. Gated Delta Networks: Improving Mamba2 with Delta Rule. *arXiv preprint arXiv:2412.06464* (2024).
- [33] Zhenyu Zhang et al. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. *Advances in Neural Information Processing Systems (NeurIPS)* (2023). arXiv:2306.14048.

A Koopman Operator Estimation

This appendix provides complete derivations for the mathematical claims made in the main text. We proceed from the basic least-squares estimator through whitening, spectral filtering, and perturbation analysis, giving explicit proofs at each step.

A.1 Least-Squares Koopman Estimator

PROPOSITION A.1 (KOOPMAN LEAST-SQUARES ESTIMATOR). Let $\{z_t\}_{t=1}^T$ be a sequence of feature vectors $z_t \in \mathbb{R}^r$. Assume an approximate linear dynamical model $z_{t+1} \approx Az_t$. The least-squares estimator

$$A^* = \arg \min_{A \in \mathbb{R}^{r \times r}} \sum_{t=1}^{T-1} \|z_{t+1} - Az_t\|^2 \quad (16)$$

is given by $A^* = MG^{-1}$, where

$$G = \sum_{t=1}^{T-1} z_t z_t^\top, \quad M = \sum_{t=1}^{T-1} z_{t+1} z_t^\top, \quad (17)$$

provided G is invertible.

PROOF. Expanding the objective:

$$\begin{aligned} O(A) &= \sum_{t=1}^{T-1} (z_{t+1} - Az_t)^\top (z_{t+1} - Az_t) \\ &= \sum_t z_{t+1}^\top z_{t+1} - 2\text{tr}\left(A \sum_t z_t z_{t+1}^\top\right) + \text{tr}\left(A \sum_t z_t z_t^\top A^\top\right). \end{aligned} \quad (18)$$

Setting $\partial O / \partial A = 0$:

$$-2M^\top + 2AG = 0 \implies A = M^\top G^{-1}. \quad (20)$$

Since $M = \sum_t z_{t+1} z_t^\top$, we have $M^\top = \sum_t z_t z_{t+1}^\top$, so $A^* = MG^{-1}$ after transposing both sides of the normal equation (equivalently, taking the derivative with respect to A in the standard matrix calculus convention where $\partial \text{tr}(AX) / \partial A = X^\top$). $\square$

REMARK 1. The lagged structure is essential: M uses (z_{t+1}, z_t) pairs, not (z_t, z_t) . If M were replaced by G , the estimator would yield $A = I$ regardless of the data, which is uninformative.

A.2 Ridge Regularization

When $T - 1 < r$ or when features are nearly collinear, G is singular or ill-conditioned. We replace G with the regularized Gram matrix

$$\tilde{G} = G + \varepsilon I, \quad \varepsilon > 0, \quad (21)$$

yielding the ridge estimator $A_\varepsilon = M\tilde{G}^{-1}$.

PROPOSITION A.2 (RIDGE AS SHRINKAGE). Let $G = \sum_t z_t z_t^\top$ have eigendecomposition $G = V\Sigma V^\top$ with eigenvalues $\sigma_1 \geq \dots \geq \sigma_r \geq 0$. Then the ridge estimator satisfies

$$A_\varepsilon = MV \text{diag}\left(\frac{1}{\sigma_1 + \varepsilon}, \dots, \frac{1}{\sigma_r + \varepsilon}\right) V^\top. \quad (22)$$

In particular, directions with small data support ($\sigma_j \ll \varepsilon$) are shrunk toward zero: the corresponding component of A_ε scales as $\sigma_j / (\sigma_j + \varepsilon) \rightarrow 0$.

PROOF. Since $\tilde{G} = V(\Sigma + \varepsilon I)V^\top$, we have $\tilde{G}^{-1} = V(\Sigma + \varepsilon I)^{-1}V^\top$. Substituting into $A_\varepsilon = M\tilde{G}^{-1}$ gives the result. The shrinkage factor for the j -th eigendirection of G is $\sigma_j / (\sigma_j + \varepsilon)$, which vanishes as $\sigma_j \rightarrow 0$. $\square$

A.3 Whitening and Spectral Equivalence

PROPOSITION A.3 (WHITENED OPERATOR). Let $\tilde{G} = LL^\top$ be the Cholesky factorization of the regularized Gram matrix. Define the whitened Koopman operator

$$A_w = L^{-1}ML^{-\top}. \quad (23)$$

Then A_w and $A_\varepsilon = M\tilde{G}^{-1}$ have the same eigenvalues.

PROOF. We show that A_w and A_ε are related by a similarity transform. Write

$$A_\varepsilon = M\tilde{G}^{-1} = M(LL^\top)^{-1} = ML^{-\top}L^{-1}. \quad (24)$$

Now observe:

$$L^{-1}A_\varepsilon L = L^{-1}(ML^{-\top}L^{-1})L \quad (25)$$

$$= L^{-1}ML^{-\top} \quad (26)$$

$$= A_w. \quad (27)$$

Since $A_w = L^{-1}A_\varepsilon L$, the two matrices are similar and therefore share the same eigenvalues. $\square$

REMARK 2. Although A_w and A_ε share eigenvalues, they differ in conditioning. The naive product $\tilde{G}^{-1}M$ (note the reversed order) is not similar to A_ε in general, and can have a different eigenspectrum. The whitened form $L^{-1}ML^{-\top}$ is the correct implementation.

A.4 Positive Definiteness of the Regularized Gram

LEMMA A.4. For any $\varepsilon > 0$, the regularized Gram matrix $\tilde{G} = G + \varepsilon I$ is positive definite and satisfies $\lambda_{\min}(\tilde{G}) \geq \varepsilon$.

PROOF. $G = \sum_t z_t z_t^\top$ is positive semidefinite since for any $u \in \mathbb{R}^r$, $u^\top G u = \sum_t (z_t^\top u)^2 \geq 0$. Therefore $u^\top \tilde{G} u = u^\top G u + \varepsilon \|u\|^2 \geq \varepsilon \|u\|^2 > 0$ for all $u \neq 0$. The minimum eigenvalue bound follows from $\lambda_{\min}(\tilde{G}) = \lambda_{\min}(G) + \varepsilon \geq \varepsilon$. $\square$

B Clebsch–Gordan Invariant Lift

B.1 Rotation Group Action on SSM Keys

The Mamba-3 rotary mechanism applies a position-dependent phase to each frequency component of the key vector. In complex notation, the j -th component of the rotated key at position t is

$$k_{t,j} = \tilde{k}_{t,j} e^{i\theta_{t,j}}, \quad (28)$$

where $\tilde{k}_{t,j} \in \mathbb{C}$ is the content-dependent base key and $\theta_{t,j} = \sum_{s=1}^t \delta_s \omega_j$ is the accumulated phase. In real coordinates, this corresponds to an $\text{SO}(2)$ rotation of each even-odd pair by angle $\theta_{t,j}$. The full rotation acts as $\text{SO}(2)^{N/2}$ on $\mathbb{R}^N$.

B.2 Degree-2 Tensor Product Decomposition

PROPOSITION B.1 (CG DECOMPOSITION FOR $\text{SO}(2)$). Let $z \in \mathbb{C}$ transform under $\text{SO}(2)$ as $z \mapsto ze^{i\theta}$. The degree-2 tensor products of z decompose into irreducible representations as follows:

- (1) The invariant (trivial representation): $z\bar{z} = |z|^2$, satisfying $|ze^{i\theta}|^2 = |z|^2$ for all θ .
- (2) The second harmonic (rotation by 2θ): z^2 , satisfying $(ze^{i\theta})^2 = z^2 e^{i2\theta}$.

PROOF. For the invariant: $|ze^{i\theta}|^2 = (ze^{i\theta})(\overline{ze^{i\theta}}) = z\bar{z}e^{i\theta}e^{-i\theta} = |z|^2$.

For the harmonic: $(ze^{i\theta})^2 = z^2e^{i2\theta}$, which transforms under the $\text{SO}(2)$ representation of angular frequency 2. $\square$

B.3 Invariance of the Feature Map

THEOREM B.2 (PHASE CANCELLATION IN THE INVARIANT LIFT). Define the invariant feature map $\psi_{\text{inv}} : \mathbb{C}^{N/2} \rightarrow \mathbb{R}^{N/2}$ by

$$[\psi_{\text{inv}}(k)]_j = |k_j|^2 = (k_j^{\Re})^2 + (k_j^{\Im})^2. \quad (29)$$

If $k_t = \tilde{k}_t \odot e^{i\theta_t}$ componentwise, then

$$\psi_{\text{inv}}(k_t) = \psi_{\text{inv}}(\tilde{k}_t) \quad (30)$$

for all θ_t . In particular, the Gram matrix $G = \sum_t \psi_{\text{inv}}(k_t)\psi_{\text{inv}}(k_t)^\top$ is independent of the position encodings $\{\theta_t\}$.

PROOF. By Proposition B.1 applied componentwise, $|k_{t,j}|^2 = |\tilde{k}_{t,j}|^2$ for each j . Therefore $\psi_{\text{inv}}(k_t) = \psi_{\text{inv}}(\tilde{k}_t)$, and any function of $\{\psi_{\text{inv}}(k_t)\}_{t=1}^T$ is independent of $\{\theta_t\}$. $\square$

COROLLARY B.3 (CONTENT-ADDRESSED GRAM MATRIX). Under the invariant lift, the inner product $\langle \psi_{\text{inv}}(k_s), \psi_{\text{inv}}(k_t) \rangle$ depends only on the content vectors $\tilde{k}_s, \tilde{k}_t$ and not on positions s, t . Thus G functions as a content-addressed index.

B.4 Harmonic Features and Equivariance

The harmonic channel retains relative phase information. In real coordinates:

$$\Re(k_j^2) = (k_j^{\Re})^2 - (k_j^{\Im})^2, \quad (31)$$

$$\Im(k_j^2) = 2k_j^{\Re}k_j^{\Im}. \quad (32)$$

Under rotation by θ , these transform as

$$\begin{pmatrix} \Re((k_j e^{i\theta})^2) \\ \Im((k_j e^{i\theta})^2) \end{pmatrix} = \begin{pmatrix} \cos 2\theta & -\sin 2\theta \\ \sin 2\theta & \cos 2\theta \end{pmatrix} \begin{pmatrix} \Re(k_j^2) \\ \Im(k_j^2) \end{pmatrix}. \quad (33)$$

The harmonic features are not invariant but are *equivariant*: they transform predictably under the group action. The full feature map $\psi(k) = [\psi_{\text{inv}}(k); \Re(k^2); \Im(k^2)] \in \mathbb{R}^{3N/2}$ combines invariant and equivariant channels, allowing the learned projection Φ to select the combination best suited to the task.

C Perturbation Analysis

C.1 Operator Perturbation Bound

THEOREM C.1 (PERTURBATION OF THE KOOPMAN ESTIMATOR). Let G, M be the true covariance matrices and let $\hat{G} = G + E_G, \hat{M} = M + E_M$ be empirical estimates with perturbations E_G, E_M . Define $\tilde{G} = G + \varepsilon I$ and $\hat{\tilde{G}} = \hat{G} + \varepsilon I = \tilde{G} + E_G$. Then for the Koopman estimators $A = M\tilde{G}^{-1}$ and $\hat{A} = \hat{M}\hat{\tilde{G}}^{-1}$:

$$\|\hat{A} - A\| \leq \|E_M\| \cdot \|\hat{\tilde{G}}^{-1}\| + \|A\| \cdot \|\tilde{G}^{-1}\| \cdot \|E_G\| \cdot \|\hat{\tilde{G}}^{-1}\| \cdot \|\tilde{G}\|. \quad (34)$$

In particular, the perturbation is controlled by $\|\tilde{G}^{-1}\| = 1/\lambda_{\min}(\tilde{G})$.

PROOF. Write

$$\hat{A} - A = \hat{M}\hat{\tilde{G}}^{-1} - M\tilde{G}^{-1} \quad (35)$$

$$= (M + E_M)\hat{\tilde{G}}^{-1} - M\tilde{G}^{-1} \quad (36)$$

$$= E_M\hat{\tilde{G}}^{-1} + M(\hat{\tilde{G}}^{-1} - \tilde{G}^{-1}). \quad (37)$$

For the second term, apply the resolvent identity. Since $\hat{\tilde{G}} = \tilde{G} + E_G$:

$$\hat{\tilde{G}}^{-1} - \tilde{G}^{-1} = -\tilde{G}^{-1}E_G\hat{\tilde{G}}^{-1}. \quad (38)$$

To verify: multiply both sides on the left by $\tilde{G}$ and on the right by $\hat{\tilde{G}}$:

$$\tilde{G}(\hat{\tilde{G}}^{-1} - \tilde{G}^{-1})\hat{\tilde{G}} = \tilde{G}\hat{\tilde{G}}^{-1}\hat{\tilde{G}} - \tilde{G} \quad (39)$$

$$= \tilde{G} - \hat{\tilde{G}} \quad (40)$$

$$= -E_G, \quad (41)$$

confirming the identity. Substituting:

$$\hat{A} - A = E_M\hat{\tilde{G}}^{-1} - M\tilde{G}^{-1}E_G\hat{\tilde{G}}^{-1}. \quad (42)$$

Taking norms and using $M = A\tilde{G}$:

$$\|\hat{A} - A\| \leq \|E_M\| \cdot \|\hat{\tilde{G}}^{-1}\| + \|A\| \cdot \|\tilde{G}\| \cdot \|\tilde{G}^{-1}\| \cdot \|E_G\| \cdot \|\hat{\tilde{G}}^{-1}\|. \quad \square$$

COROLLARY C.2 (ROLE OF $\lambda_{\min}(\tilde{G})$). Since $\|\tilde{G}^{-1}\|_2 = 1/\lambda_{\min}(\tilde{G})$, increasing the minimum eigenvalue of the regularized Gram matrix directly reduces the worst-case amplification of covariance noise into the operator estimate. This motivates logging $\lambda_{\min}(\tilde{G})$ as a stability diagnostic.

C.2 Eigenvalue Perturbation

THEOREM C.3 (BAUER-FIKE). Let $A \in \mathbb{R}^{r \times r}$ be diagonalizable with $A = V\Lambda V^{-1}$. For any eigenvalue $\hat{\lambda}$ of $\hat{A} = A + E$, there exists an eigenvalue λ of A such that

$$|\hat{\lambda} - \lambda| \leq \kappa(V) \cdot \|E\|, \quad (43)$$

where $\kappa(V) = \|V\| \cdot \|V^{-1}\|$ is the condition number of the eigenvector matrix.

PROOF. This is the classical Bauer-Fike theorem. If $\hat{\lambda}$ is not an eigenvalue of A , then $(\hat{\lambda}I - A)$ is invertible and $\hat{A}v = \hat{\lambda}v$ implies $v = (\hat{\lambda}I - A)^{-1}Ev$. Taking norms: $1 \leq \|(\hat{\lambda}I - A)^{-1}\| \cdot \|E\|$. Writing $A = V\Lambda V^{-1}$:

$$(\hat{\lambda}I - A)^{-1} = V(\hat{\lambda}I - \Lambda)^{-1}V^{-1}, \quad (44)$$

so $\|(\hat{\lambda}I - A)^{-1}\| \leq \kappa(V)/\min_j |\hat{\lambda} - \lambda_j|$. Rearranging gives $\min_j |\hat{\lambda} - \lambda_j| \leq \kappa(V)\|E\|$. $\square$

COROLLARY C.4 (COMBINED BOUND). Combining Theorem C.1 and Theorem C.3, the eigenvalue perturbation of the Koopman operator due to finite-sample estimation errors is bounded by

$$\min_j |\hat{\lambda}_j - \lambda_j| \leq \kappa(V_{A_w}) \left(\frac{\|E_M\|}{\lambda_{\min}(\hat{\tilde{G}})} + \frac{\|A\| \cdot \|\tilde{G}\| \cdot \|E_G\|}{\lambda_{\min}(\tilde{G}) \cdot \lambda_{\min}(\hat{\tilde{G}})} \right). \quad (45)$$

This makes explicit that eigenvalue stability requires both a well-conditioned Gram matrix (large $\lambda_{\min}$) and a well-conditioned eigenvector matrix of the operator (small $\kappa(V_{A_w})$).

C.3 Why Phase Nonstationarity Harms the Gram Matrix

PROPOSITION C.5 (PHASE-INDUCED CONDITIONING LOSS). Consider scalar complex features $k_t = \tilde{k}_t e^{i\theta_t}$ where $\tilde{k}_t$ and θ_t are independent, and θ_t is uniformly distributed on $[0, 2\pi)$. Let $G_{\text{raw}} = \sum_t k_t \bar{k}_t = \sum_t |\tilde{k}_t|^2$ (which is real and positive) denote the diagonal Gram entry. For the cross-covariance:

$$M_{\text{raw}} = \sum_t k_{t+1} \bar{k}_t = \sum_t \tilde{k}_{t+1} \bar{\tilde{k}}_t e^{i(\theta_{t+1} - \theta_t)}. \quad (46)$$

If $\theta_{t+1} - \theta_t$ has high variance and is weakly correlated with the content factors, the oscillatory terms $e^{i(\theta_{t+1} - \theta_t)}$ cause partial cancellation in M_{raw} , reducing $|M_{\text{raw}}|$ relative to G_{raw} . The resulting operator $A = M_{\text{raw}}/G_{\text{raw}}$ is attenuated toward zero, obscuring the true dynamics.

PROOF. In the extreme case where $\theta_{t+1} - \theta_t$ is i.i.d. uniform on $[0, 2\pi)$:

$$\mathbb{E}[e^{i(\theta_{t+1} - \theta_t)}] = 0. \quad (47)$$

Therefore $\mathbb{E}[M_{\text{raw}}] = 0$ regardless of the content dynamics, while $\mathbb{E}[G_{\text{raw}}] = \sum_t \mathbb{E}[|\tilde{k}_t|^2] > 0$. The operator estimate $A = MG^{-1}$ converges to zero rather than the true dynamics.

The invariant lift $\psi(k_t) = |k_t|^2 = |\tilde{k}_t|^2$ removes the oscillatory factor entirely, so that $M_{\text{inv}} = \sum_t |\tilde{k}_{t+1}|^2 |\tilde{k}_t|^2$ reflects the true content dynamics without cancellation. $\square$

D Spectral Filtering

D.1 Power Filter Analysis

PROPOSITION D.1 (MODE SEPARATION UNDER THE POWER FILTER). Let A_w have eigendecomposition $A_w = V\Lambda V^{-1}$ with eigenvalues $\lambda_1, \dots, \lambda_r$. The power filter applies A_w^K to a query vector w_q , which acts as $V\Lambda^K V^{-1} w_q$. Each eigenvalue is raised to the K -th power: $\lambda_j \mapsto \lambda_j^K$.

For $K = 2$:

$ \lambda $	$ \lambda ^2$
0.95	0.902
0.90	0.810
0.70	0.490
0.50	0.250
0.30	0.090

For $K = 4$:

$ \lambda $	$ \lambda ^4$
0.95	0.815
0.90	0.656
0.70	0.240
0.50	0.063
0.30	0.008

PROOF. If $A_w = V\Lambda V^{-1}$, then $A_w^K = V\Lambda^K V^{-1}$ by induction on K . The entries of Λ^K are λ_j^K , giving the tabulated values. $\square$

REMARK 3. At $K = 2$, modes with $|\lambda| \geq 0.7$ retain at least 49% of their energy, providing a wide band of usable persistent modes. At $K = 4$, only modes with $|\lambda| \geq 0.85$ retain more than 50%, which can be too aggressive when the operator has many modes in the 0.6–0.8 range that carry useful partial information.

D.2 Contractivity After Spectral Normalization

PROPOSITION D.2 (POWER FILTER CONTRACTIVITY). Let $\hat{A}_w = A_w / \max(1, \sigma_{\max}(A_w))$, where $\sigma_{\max}$ denotes the spectral norm (largest singular value). Then $\sigma_{\max}(\hat{A}_w) \leq 1$, and for any $K \geq 1$:

$$\|\hat{A}_w^K\| \leq 1. \quad (48)$$

PROOF. By construction, $\sigma_{\max}(\hat{A}_w) = \sigma_{\max}(A_w) / \max(1, \sigma_{\max}(A_w)) \leq 1$. For any matrix with spectral norm at most 1, $\|A^K\| \leq \|A\|^K \leq 1$ by submultiplicativity. $\square$

REMARK 4. This guarantees that the power filter cannot amplify any component of the query vector, preventing the gradient spikes observed when the unnormalized operator has $\sigma_{\max} \gg 1$.

D.3 Variance Restoration via SSN

Spectral normalization divides all eigenvalues by $\sigma_{\max}$, which can push persistent modes (those near $|\lambda| = 1$) below the effective filter threshold.

PROPOSITION D.3 (CONTROLLED RESTORATION). Let $\hat{A}_w$ be the spectrally normalized operator with $\sigma_{\max}(\hat{A}_w) \leq 1$. Applying a learned scalar $\gamma \in [1, \gamma_{\max}]$ yields $A'_w = \gamma \hat{A}_w$ with

$$\sigma_{\max}(A'_w) = \gamma \cdot \sigma_{\max}(\hat{A}_w) \leq \gamma_{\max}. \quad (49)$$

For $\gamma_{\max} = 1.5$ and $K = 2$:

$$\|A'_w^K\| \leq \gamma_{\max}^K = 2.25, \quad (50)$$

which is bounded and does not grow with model dimensions. The power filter remains well-behaved while eigenvalues in the range $[1/\gamma_{\max}, 1]$ can be restored to magnitudes above the filter threshold.

PROOF. $\sigma_{\max}(A'_w) = |\gamma| \cdot \sigma_{\max}(\hat{A}_w) \leq \gamma_{\max} \cdot 1 = \gamma_{\max}$. The power filter norm bound follows from submultiplicativity. $\square$

E Sequence-Max Normalization

PROPOSITION E.1 (GRAM MATRIX BOUND UNDER SEQUENCE-MAX NORMALIZATION). Let $\{z_t\}_{t=1}^T$ be feature vectors and define $\bar{z}_t = z_t / \max_s \|z_s\|$. Then $\|\bar{z}_t\| \leq 1$ for all t , and the Gram matrix $\bar{G} = \sum_t \bar{z}_t \bar{z}_t^T$ satisfies

$$\|\bar{G}\|_F \leq T, \quad \text{tr}(\bar{G}) \leq T, \quad \lambda_{\max}(\bar{G}) \leq T. \quad (51)$$

PROOF. By construction $\|\bar{z}_t\| \leq 1$, so each rank-1 term $\bar{z}_t \bar{z}_t^T$ has Frobenius norm $\|\bar{z}_t\|^2 \leq 1$ and trace $\|\bar{z}_t\|^2 \leq 1$. Summing over T terms gives the bounds. $\square$

REMARK 5. Unlike per-token ℓ_2 -normalization (which sets $\|z_t\| = 1$ for all t , inflating low-norm noise tokens to unit norm), sequence-max normalization preserves the relative norm structure within a sequence: high-norm fact-table tokens remain dominant while low-norm distractor tokens are not amplified.

F SKA Retrieval Pipeline

For reference, we state the complete SKA computation as a single pipeline, consolidating the results above.

(1) **Feature extraction.** Given the post-Mamba residual $h_t \in \mathbb{R}^d$, compute per-head features:

$$z_t = W_K h_t \in \mathbb{R}^r, \quad z_t^{(q)} = W_Q h_t \in \mathbb{R}^r, \quad v_t = W_V h_t \in \mathbb{R}^P. \quad (52)$$

(2) **Sequence-max normalization.** Let $v = \max_t \|z_t\|$. Replace $z_t \leftarrow z_t/v$ and $z_t^{(q)} \leftarrow z_t^{(q)}/v$.

(3) **Masked accumulation.** Using the action mask $m_t \in \{0, 1\}$:

$$G = \sum_{t:m_t=1} z_t z_t^\top, \quad M = \sum_{\substack{t:m_t=1 \\ m_{t+1}=1}} z_{t+1} z_t^\top, \quad C = \sum_{t:m_t=1} v_t z_t^\top. \quad (53)$$

(4) **Regularization and Cholesky.** $\tilde{G} = G + \epsilon I$, $\tilde{G} = LL^\top$ (Lemma A.4 guarantees existence).

(5) **Whitened Koopman operator.** Solve $LY = M$ and $LZ = Y^\top$ by forward substitution, then $A_w = Z^\top$ (Proposition A.3).

(6) **Spectral normalization.** Estimate $\sigma_{\max}(A_w)$ via power iteration (6 steps). Set $\hat{A}_w = A_w / \max(1, \sigma_{\max})$, then $A'_w = \gamma \hat{A}_w$ with learned $\gamma \in [1.0, 1.5]$ (Proposition D.2, Proposition D.3).

(7) **Value map.** Solve $\tilde{G}X = C^\top$, set $B_v = X^\top \in \mathbb{R}^{P \times r}$.

(8) **Query whitening.** Solve $Lw_q = z^{(q)}$ by forward substitution.

(9) **Power filter.** Compute $w_f = A'_w{}^K w_q$ (Proposition D.1).

(10) **Un-whiten and retrieve.** $z_f = Lw_f$, $\hat{y} = \eta \cdot B_v z_f \in \mathbb{R}^P$.

All operations in steps 3–9 are performed in FP32. The output $\hat{y}$ is cast back to the training precision and added to the residual stream.

G Experimental Setup

G.1 Model Architectures

We evaluate two model families across all experiments:

Mamba-3 (Baseline). A pure state-space language model built from stacked Mamba-3 blocks. Each block consists of: (i) a Layer-Norm pre-normalization, (ii) a fused input projection that produces keys $\mathbf{b}$, queries $\mathbf{c}$, values $\mathbf{v}$, step size Δ , decay a , and interpolation weight λ from a single linear map, (iii) a depth-wise 1-D convolution (kernel size 4) with SiLU activation on the value branch, (iv) a complex-valued selective state-space recurrence with Rotary Position Embeddings (RoPE) on keys and queries, computed via a chunked Structured State-Space Duality (SSD) scan in FP32 with chunk size $C=64$, and (v) a linear output projection. Each Mamba-3 block is followed by a SwiGLU MLP with expansion ratio $\frac{8}{3}$ (inner dimension rounded up to the nearest multiple of 64).

Mamba-3 + CG + SKA. The baseline augmented with Spectral Koopman Attention (SKA) modules inserted at the final two layers. SKA provides a complementary non-recurrent retrieval pathway via a learned Koopman operator. Key design choices:

- **Independent projections:** SKA uses its own key/query/value projections from the residual stream (orthogonally initialized), decoupled from Mamba’s rotary features.
- **Whitened Koopman operator:** $A_w = L^{-1}ML^{-\top}$ where L is the Cholesky factor of the ridge-regularized Gram matrix $\tilde{G} = G + \epsilon I$ ($\epsilon=10^{-3}$).
- **Scaled Spectral Normalization (SSN):** A_w is spectrally normalized via 6-step power iteration, then rescaled by a learnable $\gamma \in [1.0, 1.5]$.
- **Power spectral filter:** A_w^K with $K=2$ suppresses transient modes while preserving persistent ones.
- **Sequence-max normalization** on lifted features.

- **Ungated additive injection:** the SKA output is projected and added directly to the residual stream (no sigmoid gate, which cannot learn from ≤ 2 answer tokens per example).
- **Action masking:** only high-signal tokens contribute to the Gram and cross-covariance matrices.

G.2 Hyperparameters

Table 3 summarizes the architectural hyperparameters used in each experiment. All configurations share a state dimension $N=16$, chunk size $C=64$, SKA rank $r=24$, ridge $\epsilon=10^{-3}$, and power filter order $K=2$.

Table 3: Architectural hyperparameters by experiment. d : model dimension; H : heads; L : layers; $|V|$: vocab size; $d_{\text{head}} = d/H$; d_{ff} : SwiGLU inner dimension ($\lceil d \cdot 8/3 \rceil_{64}$).

Experiment	d	H	L	$ V $	d_{head}	d_{ff}
Tool-Calling	96	3	3	128	32	256
CoT Retrieval	96	4	4	128	24	256

For SKA-augmented models, the module is attached at layers $L-2$ and $L-1$ (the final two layers). We test two SKA scaling coefficients: $\eta=1.5$ (standard) and $\eta=2.5$ (strong) to demonstrate a dose-response relationship.

G.3 Training Configuration

All models are trained with the following protocol:

Table 4: Training hyperparameters.

Parameter	Value
Optimizer	AdamW ($\beta_1=0.9$, $\beta_2=0.95$)
Weight decay	0.01
Peak learning rate	3×10^{-4}
LR schedule	Cosine decay with linear warmup
Steps	2000-6000
Gradient clipping	Max-norm = 1.0
Batch size	16
Precision	Mixed FP16 (AMP) when GPU available

G.4 Evaluation Tasks

Experiment 1: Tool-Calling Retrieval. Sequences encode a history of multi-agent tool calls in the format `Agent: Tool→Result`, followed by a query `?Agent: Tool=`. The model must retrieve the correct result from the context. We evaluate two difficulty tiers: **EASY** (10–30 calls, $T_{\max}=400$) and **HARD** (60–120 calls, $T_{\max}=1,500$). Training runs for 2,500 steps with evaluation every 250 steps.

Experiment 2: Resource Economy. Multi-agent economy sequences where agents mine, trade, and build with gold and wood. The query requires predicting an agent’s current resource count. **EASY**: 5–15 steps, 3 agents, no noise gap ($T_{\max}=400$). **HARD**: 40–100 steps, 6 agents, noise gap of 200–400 tokens ($T_{\max}=3,000$). Training runs for 2,500 steps with evaluation every 250 steps.

Experiment 3: System Prompt Amnesia. Two sub-tasks testing retrieval of system-level variables across a long intervening gap:

- **Sub-task A (CoT Retrieval):** $n_{\text{vars}}=4$ system variables followed by a noisy chain-of-thought scratchpad. EASY: gap 10–30 tokens ($T_{\text{max}}=400$); HARD: gap 150–300 tokens ($T_{\text{max}}=3,000$).
- **Sub-task B (Specific Recall):** $n_{\text{vars}}=6$ system variables followed by confusing distractors. EASY: gap 10–30 tokens ($T_{\text{max}}=500$); HARD: gap 80–200 tokens ($T_{\text{max}}=2,500$).

Training runs for 6,000 steps with evaluation every 250 steps. All models in this experiment use $L=4$ layers and $H=4$ heads.

G.5 Recurrence Implementation

The selective state-space recurrence $\mathbf{H}_t = \alpha_t \mathbf{H}_{t-1} + \beta_t \mathbf{U}_{t-1} + \gamma_t \mathbf{U}_t$ is computed via a chunked SSD algorithm (following Mamba-2) with chunk size $C=64$. The intra-chunk computation uses lower-triangular decay matrices via segment-sum, while boundary states are propagated across chunks via a short sequential scan over T/C steps. All scan-critical arithmetic (log-space cumulative sums, decay matrices, boundary propagation) is performed in FP32 to ensure numerical stability; only the final output is cast back to the training precision.

H Additional Results

H.1 Ablation: Removing the Action Mask

Section ?? introduced the action mask as a form of structured inductive bias that restricts SKA’s covariance accumulation to fact-bearing tokens, preventing dilution from uninformative distractor spans. A natural question is whether this mask is necessary for SKA to function, or whether the spectral filtering mechanism is robust enough to isolate retrievable content even when distractor tokens contribute to the covariance statistics. Figure 1 answers this by training CG+SKA on the short Tool Trace tier with the action mask set to $m_t = 1$ for all input tokens, so that distractor tokens and fact-bearing tokens contribute equally to G, M, and C. Both the log loss and accuracy curves closely track those of the action-masked configuration, and the model converges to comparable accuracy.

H.2 Additional plots for System Prompt CoT Retrieval task

Figure 2 shows training curves for all three model variants on the Easy and Hard tiers of the CoT Retrieval sub-task. On the Easy tier (left panel), both CG+SKA configurations rise sharply from near-chance accuracy to 100% within the first 500 steps and remain saturated for the remainder of training. Mamba-3, despite training for the full 2,000 steps, fluctuates between 10–20% accuracy with no discernible upward trend, confirming that the recurrent state cannot learn to preserve system-prompt bindings even across short scratchpad gaps.

The Hard tier (right panel) presents a more demanding setting, with 150–300 CoT steps interposed between the system prompt and the query. The CG+SKA variants exhibit a brief delay relative to the Easy tier—reaching saturation at roughly 750–1,000 steps rather than 500—but converge to the same perfect accuracy. This

delay is expected: longer sequences produce larger covariance matrices with more distractor contribution, requiring slightly more training for the Koopman operator’s spectral structure to stabilize. Mamba-3 again shows no learning signal, remaining at or below 16% throughout. The flat Mamba-3 curves on both tiers, contrasted with the rapid SKA convergence, provide further evidence that the retrieval gain is architectural rather than a consequence of additional training signal or capacity.

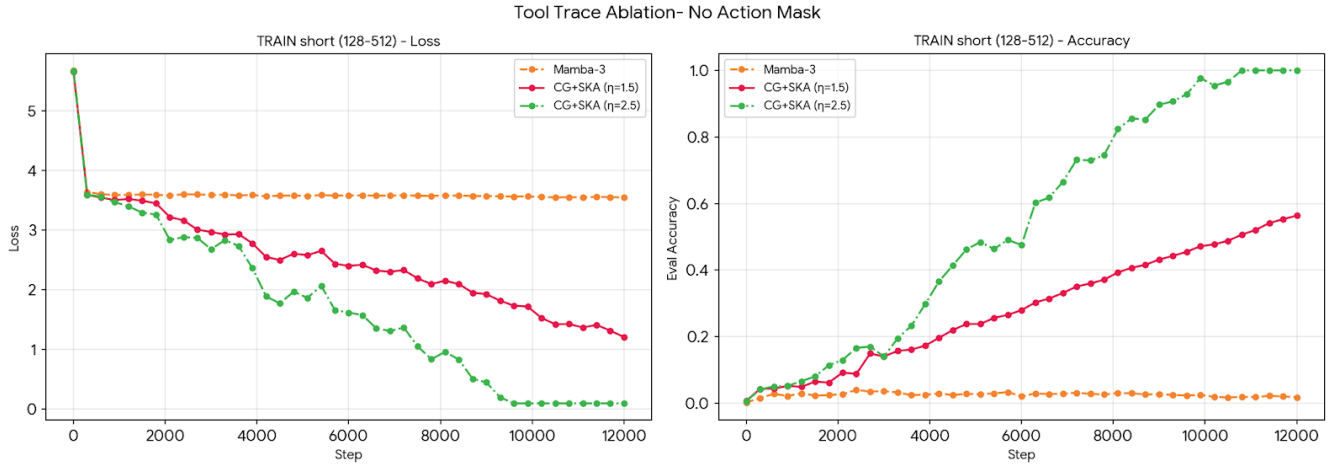


Figure 1: Training with action mask set to 1 for all inputs on the Tool Trace Delayed Commit tasks (short). We find that CG+SKA works fine without the action mask. The x-axis is for training steps and y-axis for log loss (left) and accuracy (right).

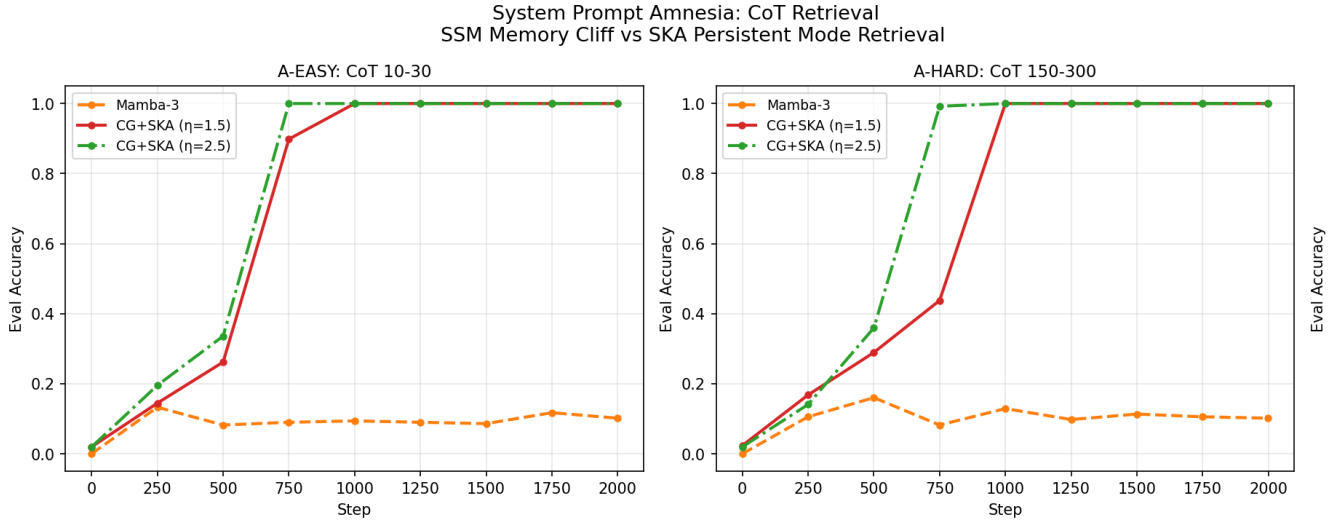


Figure 2: A Training run for 400k parameter Mamba-3 variants over the easy and hard System Prompt CoT Retrieval benchmark. The x-axis represents training iterations and y-axis represents training iterations and y-curve evaluation accuracy. Mamba-3 is unable to learn on this task.